Master Product & System Architecture Document

Project Name: FluxHeal — Autonomous Self-Healing ETL/ELT Data Pipeline

Domain: Clean-Tech Data Infrastructure / Green Data Engineering / Industrial IoT & ESG Analytics

Document Type: General Product, Architecture & Engineering Blueprint (v1.0)

---

1. Executive Summary & Product Vision

Modern Clean-Tech ecosystems—including renewable energy smart grids, precision agriculture networks, EV charging infrastructures, and corporate carbon-accounting platforms—depend on continuous, high-velocity data streams. However, traditional Extract-Transform-Load (ETL) and Extract-Load-Transform (ELT) pipelines are inherently brittle. A minor firmware update on a solar inverter, a malfunctioning moisture sensor sending `null` values, or an unexpected API rate limit can crash an entire data pipeline.

This fragility creates two severe problems:

1. **Operational Blindness:** Sustainability and grid-optimization algorithms lose real-time visibility, leading to wasted water, suboptimal energy routing, and delayed emissions reporting.
2. **Dirty Cloud Compute (High Scope 3 Emissions):** When pipelines fail mid-flight, data engineering teams must manually fix scripts and execute massive historical data backfills. Re-processing millions of rows wastes cloud server CPU cycles, increases data-center electricity consumption, and inflates the carbon footprint of the software infrastructure itself.

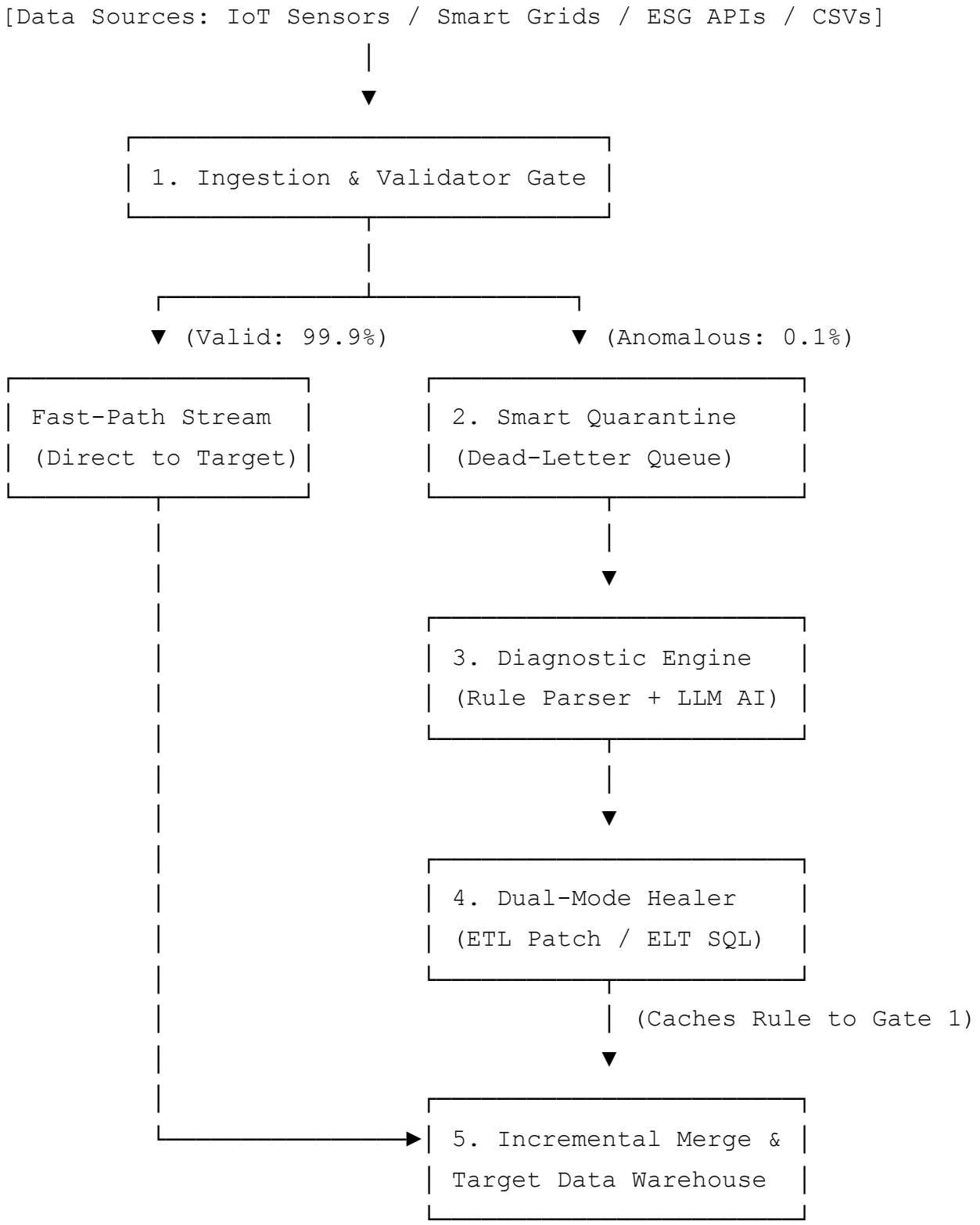
The Product Vision:

We are engineering an autonomous, zero-downtime **Self-Healing ETL/ELT Data Pipeline Platform**. Instead of halting workflows when encountering schema drift, corrupt sensor telemetry, or broken SQL transformations, the system isolates only the defective records in milliseconds, diagnoses the root cause using a hybrid Rule + AI Diagnostic Engine, synthesizes a live patch, heals the data, and caches the resolution rule for future payloads—eliminating both pipeline downtime and redundant cloud compute waste.

---

2. Core System Architecture & The 5-Stage Autonomous Loop

As a product and engineering system, the platform operates on a continuous, deterministic-first, AI-augmented feedback loop.



Stage 1: Multi-Source Ingestion & Edge Validator Gate

- **Function:** Ingests real-time streaming payloads (MQTT, WebSockets, Kafka/Webhook streams) and batch sources (REST APIs, CSV/JSON dumps).
- **Sub-Millisecond Check:** Every incoming payload passes through an in-memory Schema & Quality Validator that checks structural integrity, column presence, data types, and physical boundary limits (e.g., ambient temperature cannot be `-9999°C`).
- **Hot-Rule Cache:** Checks if a previously learned "healing transformation" already exists in memory; if so, it applies the fix instantly at native code speed without invoking the AI engine.

Stage 2: Zero-Downtime Micro-Quarantine (Dead-Letter Isolation)

- **Function:** Separates healthy data from broken data at the individual record/row level.
- **Core Principle:** If 10,000 smart-meter readings arrive and 3 contain corrupted strings, 9,997 records bypass the healer and flow directly into the warehouse. Only the 3 defective payloads are routed to the **Quarantine Store** along with their execution context and error stack trace.

Stage 3: Hybrid Diagnostic Engine (Root-Cause Classification)

- **Function:** Analyzes quarantined payloads and failed transformation logs to classify the exact anomaly into one of four foundational buckets:

1. **Schema Drift:** Renamed fields, newly added nested JSON keys, or reordered CSV columns.
2. **Data Corruption / Sensor Anomaly:** Missing values (`null` / `NaN`), out-of-bounds hardware spikes, or truncated JSON strings.
3. **Format & Type Mismatch:** Unit strings attached to numbers (`"45.2 kW"` instead of `45.2`), incompatible timestamp formats, or boolean encoding shifts.
4. **Warehouse Query / Runtime Fault:** SQL syntax failure, division-by-zero in aggregations, or upstream API rate limiting (`HTTP 429`).

Stage 4: Dual-Mode Autonomous Healing (ETL vs. ELT)

The platform natively supports both pipeline paradigms:

- **ETL Mode (In-Flight Stream Healing):** Before data lands in the database, the Healer Engine dynamically generates a transformation function (Regex extractor, JSON key remapper, or Time-Series Imputation calculator) to sanitize the payload in memory.
- **ELT Mode (Warehouse SQL Patching):** When raw data is already loaded into a staging table and a scheduled SQL transformation fails, the ELT Agent parses the database error log, inspects the staging table schema, rewrites the broken SQL query (e.g., adding `COALESCE`, `NULLIF`, or updated column aliases), and executes a dry-run validation.

Stage 5: Incremental Checkpoint Replay & Rule Caching

- **Function:** Re-injects the healed records into the main target table using idempotent upserts (`MERGE INTO`) without re-running the entire batch.
- **Self-Learning Loop:** Once a fix is verified, the generated mapping or regex rule is compiled and pushed back to **Stage 1 (Hot-Rule Cache)** so subsequent identical errors are healed deterministically in `<2ms` with zero LLM token cost or compute overhead.

---

3. Comprehensive Feature Matrix

Module A: Autonomous Data Healing Features

Feature Name	Technical Mechanism	Real-World Problem Solved
Semantic Schema-Drift Resolver	Uses Levenshtein distance + LLM semantic embedding to match unknown incoming keys (`active_pwr_w`) with target schema columns (`power_watts`), validating against historical value distributions.	Firmware updates on IoT gateways or third-party ESG APIs renaming JSON keys without prior notice.
Time-Series Sensor Imputation	Maintains a sliding window buffer (e.g., last 20 readings per `device_id`). Replaces `null` or extreme outlier spikes using Linear Interpolation or Exponential Moving Average (EMA).	Loose sensor wiring or weather interference causing temporary `-9999` or `NaN` spikes in smart agriculture or grid monitors.
Autonomous Format & Unit Normalizer	Synthesizes dynamic regex and type-casting parsers when numeric fields receive string-contaminated inputs (e.g., stripping `" V"` , `" kWh"` , or normalizing mixed ISO/POSIX dates).	Integrating multi-vendor smart meters that format units and timestamps differently.
Self-Patching ELT SQL Agent	Intercepts database engine exceptions (`ColumnNotFound` , `DivideByZero` , `InvalidCast`), introspects `INFORMATION_SCHEMA` , and patches the SQL AST (Abstract Syntax Tree) automatically.	Nightly sustainability aggregation jobs crashing at 99% completion due to a single zero-value denominator.
Adaptive API Backoff & Circuit Breaker	Detects upstream `HTTP 429/503` errors, pauses polling for that specific connector, calculates optimal retry windows via	External weather or carbon-intensity APIs throttling requests during peak hours.

Feature Name	Technical Mechanism	Real-World Problem Solved
	exponential backoff + jitter, and resumes from the last cursor.	

Module B: Clean-Tech & Green Compute Features

Feature Name	Technical Mechanism	Sustainability / Clean-Tech Impact
Green Compute & Carbon Telemetry Engine	Tracks rows healed in-flight vs. total dataset size. Calculates avoided CPU-seconds, memory I/O, kWh saved, and equivalent gCO_2e prevented by avoiding full-table backfills.	Quantifies the direct environmental ROI of the software for hackathon judges and enterprise ESG reports.
Carbon-Aware ELT Batch Scheduler	Integrates with grid carbon-intensity signals (e.g., Electricity Maps / WattTime API). Defers non-urgent heavy ELT warehouse aggregations to hours when local renewable energy generation is highest.	Shifts heavy data-center compute workloads to green energy windows, directly lowering Scope 3 cloud emissions.
Unified ESG & Telemetry Lineage Tagging	Appends an immutable `_audit_meta` JSON column to every healed record containing `is_healed`, `original_raw_value`, `healing_method`, and `confidence_score`.	Ensures 100% regulatory compliance and auditability for carbon accounting and environmental reporting.

Module C: Product Experience, UI & Operator Control Features

Feature Name	Product Design & UX Behavior	User / Demo Value
Interactive Pipeline Topology Canvas	Node-based live visual graph (`Source` ➔ `Validator` ➔ `Quarantine` ➔ `AI Healer` ➔ `Warehouse`) with real-time animated packet flow and health status indicators.	Gives engineers and non-technical stakeholders instant situational awareness of pipeline health.
Confidence-Gated Human-in-	Configurable autonomy slider: Fixes with Confidence $\ge 90\%$ auto-deploy; fixes	Prevents hallucinated schema mappings on mission-critical

Feature Name	Product Design & UX Behavior	User / Demo Value
the-Loop (HITL)	between $60%-89%$ trigger a "1-Click Approve/Reject Diff" card in the UI and Telegram/Slack.	financial or compliance tables.
Live Chaos / Fault Injection Studio	Built-in simulation drawer allowing operators (or hackathon judges) to inject live anomalies (*Rename Column*, *Inject Sensor Spike*, *Send Corrupt Units*, *Break SQL*) with one click.	Turns an invisible backend infrastructure tool into a highly visual, interactive, and memorable live product experience.

---

4. End-to-End Operational Workflows & Data Mutation Blueprint

To illustrate how data transforms inside the engine, below is the exact payload lifecycle across our primary failure modes:

Scenario A: Schema Drift + Unit Contamination (ETL Stream Mode)

1. **Incoming Raw Payload (Faulty):**

```
{
  "device_id": "SOLAR-INV-09",
  "ts": "02/11/2026 14:22:00",
  "output_kilowatts": "64.8 kW",
  "grid_freq": 50.1
}
```

2. **Expected Target Schema:**
   `device_id (VARCHAR)`, `timestamp (ISO-8601)`, `power_kw (FLOAT)`, `frequency_hz (FLOAT)`
3. **Diagnostic Detection:**

- Missing required fields: `timestamp`, `power_kw`, `frequency_hz`.
- Unrecognized fields present: `ts`, `output_kilowatts`, `grid_freq`.
- Type mismatch on `output_kilowatts`: String `"64.8 kW"` instead of `FLOAT`.

4. **Autonomous Healer Action:**

- Maps `ts` ➔ `timestamp` and converts `DD/MM/YYYY HH:mm:ss` to ISO format.
- Maps `output_kilowatts` ➔ `power_kw` and applies regex `parseFloat(val.replace(/[^0-9.-]+/g,""))`.
- Maps `grid_freq` ➔ `frequency_hz`.

5. **Final Healed Output Written to Warehouse:**


```
{
  "device_id": "SOLAR-INV-09",
  "timestamp": "2026-11-02T14:22:00Z",
  "power_kw": 64.8,
  "frequency_hz": 50.1,
  "_healed_meta": {
    "is_healed": true,
    "anomalies_fixed": ["SCHEMA_DRIFT", "UNIT_STRING_STRIP", "DATE_FORMAT
    "confidence": 0.98,
    "compute_saved_ms": 1420
  }
}
```


---

5. Full-Stack Technical Blueprint (Recommended Architecture)

Designed for rapid prototyping, high concurrency, and a standout live demonstration:

1. Frontend & Product UI Layer

- **Framework:** React.js / Next.js with TypeScript and Tailwind CSS.
- **Visualization Engine:** HTML5 Canvas / React Flow for real-time node topology animations; Recharts / Chart.js for live telemetry & Carbon Savings metrics.
- **Real-Time Sync:** Native WebSockets (`Socket.io` or raw `ws`) for sub-second streaming of pipeline states, quarantine alerts, and live terminal logs.

2. Backend Ingestion, Orchestration & Healer Layer

- **Runtime:** Node.js (TypeScript/Express) for high-throughput asynchronous event streaming + Python microservice (FastAPI) for statistical time-series imputation and AI agent orchestration (or unified full-stack Node/Python architecture).

- **Stream & Queue Management:** In-memory Ring Buffer + Redis / SQLite-backed Dead-Letter Queue (Quarantine Store) for zero-latency isolation.
- **Rule Caching Engine:** In-memory compiled JavaScript/Python transformation map (`Map<SourceSignature, TransformFn>`) ensuring cached fixes execute in `<1ms` .

3. Diagnostic & AI Intelligence Layer

- **Tier-1 Deterministic Engine:** Fast schema diffing, statistical Z-score / IQR outlier detection for sensor streams, and Regex pattern matchers.
- **Tier-2 LLM Agent (Fallback for Novel Drift/SQL Patching):** Structured JSON-mode LLM API calls (Gemini 2.5 Flash / OpenAI) prompted strictly with schema metadata and sample data headers (never sending bulk rows, preserving privacy and speed).

4. Storage & Data Warehouse Layer

- **Target Warehouse & Staging:** PostgreSQL (with TimescaleDB extension or DuckDB for fast analytical ELT queries).
- **Audit & Lineage Store:** Dedicated `quarantine_logs` and `healing_rules_registry` tables storing raw payloads, applied patches, timestamps, and green-compute savings metrics.

---

6. Data Governance, Safety & Rollback Architecture

A critical question from technical judges and enterprise buyers is: *"What if the AI heals the data incorrectly?"* The architecture enforces three strict safety guardrails:

1. **Non-Destructive Raw Preservation:** The original defective payload is never overwritten or deleted; it is stored verbatim in the `_healed_meta.raw_payload` audit log.
2. **One-Click Rule Rollback:** Every autonomous patch receives a `rule_version_id` . If an operator notices an inaccurate mapping, clicking **"Revert Rule"** immediately restores the raw values for affected rows and blacklists that specific transformation.
3. **Strict Confidence Thresholding:** Structural changes (like dropping columns or altering financial/carbon credit totals) automatically require Human-in-the-Loop approval if confidence falls below the safety threshold.